

Student Name: Rishaan Abrol
Email Address: rishaan.abrol15@gmail.com

Registration No: [Enter Registration Number]
Project Repository: github.com/rishaanabrol/Vellum

1. Problem Statement

Modern students and researchers increasingly rely on Generative AI and Large Language Models (LLMs) to synthesize dense academic course materials, problem sets, and textbooks. However, general-purpose LLM chat interfaces present three critical limitations in academic settings:

- **Hallucination & Lack of Verifiability:** AI models routinely produce plausible-sounding yet factually inaccurate statements without providing auditable, page-level citations or mathematical proof.
- **The Speculation Trap (No Refusal Cutoff):** When course materials do not contain the answer, conventional chatbots speculate using general pre-training data rather than admitting the answer is absent from the syllabus, misleading students.
- **Context Loss & Prompt Injection Vulnerability:** General interfaces fail to isolate user-uploaded documents from core instructions, risking syllabus confusion or adversarial prompt injection.

2. Why This Problem Statement Was Chosen

Academic study demands strict intellectual accountability: *"Don't just answer the student; show them exactly where the answer came from."*

"When university students prepare for examinations or technical coursework, ungrounded speculation is worse than no answer. A student needs deterministic proof, exact equation derivations, and the ability to verify original pages with confidence."

- **Pedagogical Rigor:** In STEM courses (physics, linear algebra, engineering), answers must cite exact formula derivations and test parameters rather than hand-wavy approximations.
- **Verifiable Evidence:** Combining conversational intelligence with high-resolution PDF page rasterization allows immediate confirmation of underlying diagrams, charts, and equations.
- **Editorial Focus:** Designed with an academic, print-monograph reading aesthetic that promotes focused, distraction-free scholarship.

3. Approach / Proposed Solution

Vellum implements an end-to-end, deterministic **Retrieval-Augmented Generation (RAG)** pipeline specifically engineered around page provenance, high-resolution rendering, and strict citation verification:

```
[PDF / TXT Course Materials]
    ↓
core/document_processor.py → Page-aware extraction, noise cleanup, scanned page detection
    ↓
core/chunker.py → Paragraph-aware chunking with overlap & page provenance
    ↓
core/embeddings.py → Google gemini-embedding-001 (RETRIEVAL_DOCUMENT / RETRIEVAL_QUERY)
    ↓
core/vector_store.py → ChromaDB persistent vector database with SHA-256 deduplication
    ↑ (Vector Similarity Search)
core/retriever.py → Top-K candidate extraction, hybrid lexical boost, thresholding
    ↑
core/llm.py (Rewriter) → Conversational follow-up rewriting (gemini-3.5-flash-lite)
    ↑
[User Question]
```

3. Architectural Highlights (Continued)

- **Page-Aware Ingestion:** PyMuPDF (fitz) extracts document text page-by-page while tracking character density to flag scanned or low-quality pages.
- **Paragraph-Preserving Chunking:** Content is segmented along natural semantic paragraphs ($\backslash n \backslash n$) with controlled overlap, ensuring mathematical formulas and proofs stay intact.
- **Dual-Stream Hybrid Retrieval:** Combines dense semantic vector search (gemini-embedding-001) with lexical keyword matching, capturing abstract concepts while preserving exact variable names and theorem definitions.
- **Deterministic Refusal Cutoff:** If retrieved chunks score below relevance threshold ($\text{MIN_RELEVANCE} = 0.65$), the LLM generation step is bypassed entirely to prevent hallucination.
- **Automated Citation Validation:** Regex post-processors parse bracketed citations ([1], [2]), verifying every citation maps to retrieved evidence chunks and stripping hallucinated markers.
- **Page Evidence Proof:** Original PDF pages are rendered on demand in high resolution within the Evidence panel, providing visual verification of charts, proofs, and figures.

4. Technologies & Tools Used

Layer / Component	Technology / Library	Purpose & Architectural Role
Frontend & UI	Streamlit (v1.65+)	Continuous 3-column academic desktop interface, reactive chat timeline, session state
Typography & Styling	CSS3, Cormorant Garamond, Manrope	Print-monograph parchment styling, custom CSS bridge, flex layout
Language Model	Google Gemini (gemini-3.5-flash-lite)	High-throughput query rewriting, document insights, and grounded response generation
Embedding Engine	Google gemini-embedding-001	High-dimensional semantic text embeddings with task-specific modes
Vector Database	ChromaDB	Persistent on-disk vector store with cosine distance indexing & metadata filters
PDF Processing	PyMuPDF (pymupdf / fitz)	Page-level extraction, scanned page detection, and high-DPI page rendering
Configuration & Secrets	Python-Dotenv & Streamlit Secrets	Secure API key management across local development and Streamlit Community Cloud
Testing & QA	PyTest (43 tests)	Comprehensive automated test suite covering chunking, embeddings, retrieval, and UI

5. Key Features & Functionality

Continuous 3-Column Desk Organized desktop keeping document library, conversation timeline, and evidence citations simultaneously accessible without context switching.	Automated Document Insights Generates high-yield summaries and 3 clickable starter study questions tailored to uploaded problem sets, lecture slides, or syllabi.
Context-Aware Query Rewriting Follow-up queries (e.g. <i>"Explain that step in simpler terms"</i>) are automatically rewritten into self-contained standalone search queries using past context.	Visual Proof & Page Render The Evidence panel renders the exact rasterized PDF page image, letting students verify original layout, diagrams, and equations visually.
Step-by-Step Solutions Formulates math problems (matrices, eigenvalues, wave equations) with complete step-by-step LaTeX solutions and verified bracketed citations [1], [2].	Notes Export & Cloud Ready One-click export of chat sessions into formatted Markdown study notes. Built-in support for Streamlit Cloud deployment with encrypted secrets management.